SAVORA

Food Ordering System with AI-Powered Menu Search

"Something spicy and vegetarian under 200 rupees." A customer types what they feel like eating; the system returns available dishes ranked by how well they actually match.

Two roles

Customer ordering + admin kitchen and menu control

Hybrid AI search

Rules + embeddings + LLM rerank, with an offline fallback

Enforced workflow

Placed → Confirmed → Preparing → Ready → Picked Up

56 backend tests

RBAC, state machine, pricing, AI degradation ladder

Biplab Kar · AI Software Engineer assignment · KPi-Tech Services

System architecture

Routers do HTTP · services do the thinking · models do storage

React 18 + Vite

Customer: menu by category · AI search · cart · order tracking
Admin: dashboard · orders board · menu CRUD — guarded by role-aware routes

▼ fetch + Bearer JWT (Vite proxies /api in dev — no CORS in the demo path)

FastAPI

/auth /menu /search /orders /dashboard
deps.py → get_current_user → require_admin | require_customer
services: constraints · lexical · gemini · ai_search · order_state

▼ SQLAlchemy 2.0

SQLite · savora.db

users · menu_items (+ cached embedding) · orders · order_items · order_events

Google Gemini

text-embedding-004
gemini-2.0-flash

Called over plain REST.
Every call returns None
on failure — never raises
into a request.

Why SQLite

The evaluation is a live demo on my laptop. A docker-compose that has to be healthy five minutes before the call is risk without benefit at this data volume. DATABASE_URL is the only change to move to Postgres.

8 backend dependencies · 3 frontend dependencies

The AI search pipeline

Four stages. Each one degrades on its own instead of taking the search down.

“something spicy and vegetarian under 200 rupees”

1 Constraints

Regex rules first, Gemini second. On conflict the rules win.

```
max_price = 200
require = spicy, vegetarian
```

↓ no LLM → rules only

→

2 Candidate set

Hard filters in SQL: price, diet, allergens, availability.

The model never gets to override a stated budget.

↓ always runs

→

3 Recall

Cosine over embeddings cached on each row, fingerprinted.

One embedding call per search — the query.

↓ no vectors → BM25

→

4 Rerank

Gemini scores the top 12 and writes a one-line reason.

Blend 0.7 × LLM + 0.3 × retrieval.

↓ no rerank → recall order

Hard constraints never reach the model

Price, diet and allergens are SQL predicates. A test proves an LLM trying to widen ₹200 → ₹5000 is ignored.

Soft preferences only steer ranking

“Light”, “not fried” on a 40-item menu would empty the results if hard-filtered. They rank; they do not exclude.

The response says which path ran

search_mode, degraded and notes are part of the contract and shown in the UI. A silent downgrade is the worse bug.

End-to-end workflow

Search → cart → order → kitchen → dashboard, with the state machine enforced server-side

Forward-only · cancellable up to Preparing · terminal states accept nothing · illegal transition = 422 naming what IS allowed · every move appended to order_events

Customer

- Searches in plain English, sees a match % and the reason
- Cart sends ids and quantities only — never prices
- Order card polls every 8s and shows a live progress rail

Admin

- Orders board renders buttons from the server's allowed_transitions
- Menu CRUD; deleting a dish that appears on past orders is a 409
- Dashboard: revenue, AOV, orders by status, popular items, by hour

Guarantees

- Prices re-read server-side; order lines snapshot name and price
- A customer fetching another customer's order gets 404, not 403
- Unavailable dishes never enter search results or a new order

If I had another week

Alembic migrations · refresh tokens with a shorter TTL · rate-limit + cache the public search endpoint · move recall into the DB (sqlite-vec / pgvector) · idempotency key on POST /orders · a labelled query set so search quality is a number, not a vibe